

CEAS Co-op Reflection System: D1 Detailed Design

1. Header, scope, and conventions

Project title: CEAS Co-op Reflection System

Goal statement (same wording as D0): Let CEAS co-op students reflect on each completed rotation using rotation-specific questions, give assigned advisors a clear summary with workplace supervisor observations, and keep each reflection private to the student and their assigned advisor.

Team members: Priyanshi Kulshrestha, Mbathio Lo, Maria Malik, Aradana Nair, Hinna Parwez

Scope: This document details C7 Reflection Database and C2 Access Control Service; C1 Web Front End, C3 Reflection Service, C4 Prompt Configuration Store, C5 Summary Generator, and C6 Supervisor Evaluation Service are deferred to D2.

Conventions for every figure in this document:

- **Boxes:** A solid box with a bold header is a table (entity) that the team builds in C7. A dashed box is an external system we read from but do not own (E2 University Records). The header is the entity name; the lines under it are the attributes that matter for behavior, not every column.
- **Lines:** A line between two boxes is a relationship. A small label near each end gives its cardinality. A relationship is read from either end: "one student has many reflections" and "each reflection belongs to exactly one student."
- **Keys:** PK marks the primary key. FK marks a foreign key (the line it follows leads to the table it points at). UQ marks a unique constraint. IDX marks a field with an index that is not already a key.
- **Cardinality:** Each end shows the minimum and maximum count of rows on that side: 1 means exactly one, 0..1 means zero or one, 0..* means zero or more, and 1..* means one or more.
- **Requirement IDs:** US-01 to US-04, UC-01, UC-02, and AC-01.1 to AC-02.3 come from our Assignment 4 requirements. D0 component names (C1 to C7, E1, E2) and interface IDs keep the meaning they have in D0.

2. Data model, the D1 diagram

C7 uses a relational database with eight tables. The tables are split so that reflections, supervisor evaluations, and advisor assignments never share a row, which is what keeps each reflection private to the student and their assigned advisor.

Decision	What we chose and why	Requirements
	save fails. A key-value or document store would move these checks into our own code.	
Answers are rows in REFLECTION_RESPONSE, not columns on REFLECTION	Prompt sets differ by rotation and can change in C4, so the number of answers varies. One row per answer needs no schema change when prompts change. The category field (accomplishment, challenge, or goal) is copied at save time, so C5 can find the three summary fields without asking C4.	US-01, US-02, AC-01.1
ADVISOR_ASSIGNMENT is its own entity (many-to-many)	A student can change advisors over time and an advisor has many students. Start and end dates are stored, so "currently assigned" is a date check and past assignments stay on record. Rows are a local copy refreshed from E2.	US-04, AC-02.1, AC-02.3
One reflection per placement and rotation	A student writes one reflection per completed rotation, so STUDENT to REFLECTION is one-to-many with a limit. The unique pair (placement_id, rotation_number) on REFLECTION enforces "no submitted reflection exists for this placement and rotation."	UC-01, AC-01.2
student_id is stored on REFLECTION even though PLACEMENT has it	The owner check (a student reads their own reflection) becomes one indexed comparison. The value is never edited after the save and is checked against PLACEMENT.student_id when the reflection is saved.	UC-02 A1, AC-02.2
SUPERVISOR_EVALUATION has no key to REFLECTION	It links to PLACEMENT only, so submitting an evaluation gives a supervisor no path to reflection content. C6 needs insert rights on this table and no read rights on the reflection tables.	US-03, US-04

Decision	What we chose and why	Requirements
rotation_source is an attribute, not an entity	It holds one of two values per reflection (university or student_supplied), so a separate table adds nothing.	UC-01 A1, AC-01.3
ACKNOWLEDGMENT is an entity, not a yes/no on STUDENT	The data-use notice is acknowledged per notice version, so one student can have several acknowledgments over time. The unique pair (student_id, notice_version) stops duplicates.	UC-01 preconditions
Indexed fields	university_id on STUDENT and ADVISOR (every request starts from the sign-in identity E1 returns); student_id on REFLECTION (owner check); (student_id, advisor_id) on ADVISOR_ASSIGNMENT (advisor check); the unique (placement_id, rotation_number) pairs (duplicate check); reflection_id on REFLECTION_RESPONSE (load all answers for one reflection). Free-text fields are not indexed because nothing searches them, and each extra index slows saves.	AC-01.2, AC-02.1, AC-02.2

3. Core algorithms

Two computations break the product if they are wrong. The access decision in C2 shows a private reflection to the wrong person if it is wrong (US-04). The save in C7 stores a duplicate or half-finished reflection if it is wrong (UC-01). Neither is slow work at our size, so the risk is correctness, not speed.

Assumed data size: about 500 students with 3 rotations each gives about 1500 reflections and about 6000 answers, with about 5 advisors. At 100 times that, there are 15,000 reflections and 600,000 answers.

Algorithm 1: Reflection access decision (C2)

1. What it is and the problem it solves. For one signed-in user and one reflection ID, C2 decides allow or deny. It enforces the privacy goal: only the owner and the owner's currently assigned advisor can read a reflection (US-04, UC-02). Supervisors, IT staff, other students, and unassigned advisors get no reflection content.

2. Inputs and outputs.

Name	Type	Notes
actor_university_id (input)	string, 1 to 64 characters	Comes from the E1 sign-in, never from the request body
reflection_id (input)	string, UUID	Comes from the URL
today (input)	date, YYYY-MM-DD	Server clock in America/New_York (assumption)
decision (output)	"allow" or "deny"	
reason (output)	owner, assigned_advisor, not_authorized, assignment_unconfirmed, or backend_unavailable	Application log only. The caller sees the same denied response for not_authorized and for an unknown ID
reflection (output)	object or null	Present only when the decision is allow

The steps, in order:

1. Find the actor in STUDENT or ADVISOR by university_id. If there is no match, deny. Supervisors and IT staff end here.
2. Read the student_id of the reflection by reflection_id. If the reflection does not exist, deny with the same response as for a reflection the actor may not read, so nobody can learn which IDs exist (AC-02.3).
3. If the actor is a student and the reflection's student_id is the actor's own, allow (AC-02.2).
4. If the actor is an advisor, look for an ADVISOR_ASSIGNMENT row for that student and advisor where start_date is on or before today and end_date is empty or on or after today. If the row exists and checked_at is less than 15 minutes old (assumption), allow (AC-02.1).
5. Otherwise ask E2 whether the advisor is assigned today. If yes, update or insert the row and allow. If no, close the row (if there is one) and deny. If E2 cannot be reached, deny with assignment_unconfirmed.
6. Only after an allow, load the reflection and its answers. Content is never read before the decision.

3. Expected complexity. Steps 1 to 4 are three indexed lookups, each $O(\log n)$. At 9,000 reflections that is about 13 steps per lookup, and at 900,000 it is about 20. The database time is a few milliseconds in both cases, so the 100 times difference does not matter and we will not tune it. The slow part is the call to E2 (a network call to another

system), which is why step 4 uses the local copy. The D0 working target of under 2 seconds applies to the whole request.

4. Why this one and not the alternatives.

- *Ask E2 on every read*: always current, but every read waits on E2 and fails when E2 is down.
- *One advisor_id on STUDENT*: simpler, but it loses history and cannot handle a change of advisor or two advisors.
- *Any user with an "advisor" role can read*: easier, but US-04 asks for an assigned relationship, not a role.
- *Each service checks access on its own*: rejected in the D0 decision log, because the rules could drift apart.

The chosen design keeps one check in one place, with a short-lived local copy to keep reads fast and to ride out a short E2 outage. The trade-off: after an advisor is unassigned, access can last up to 10 minutes.

5. Edge cases.

- Missing or malformed actor ID or reflection ID: deny with a generic invalid-request error. No reflection content is read.
- Unknown reflection ID, or a real one the actor may not read: the same denied response (AC-02.3).
- Assignment ended before today, or starts after today: deny. An end_date equal to today still allows access.
- Two current advisors for one student: both are allowed, because the model allows it.
- Duplicate assignment rows after a refresh: harmless, since the check only asks whether a matching row exists, and a refresh updates a matching row instead of inserting a new one.
- A user who is both a student and an advisor: the owner check covers their own reflections and the advisor check covers everyone else's.
- E2 down with a fresh local row: allow. E2 down with a stale or missing row: deny, which follows the D0 rule that an unconfirmed assignment means no access.
- Database down: deny and return no content, which follows D0.
- Ties and ordering: not applicable, because the result is a yes or no.

Algorithm 2: Save a reflection exactly once (C7)

1. What it is and the problem it solves. When C3 submits a completed reflection, C7 stores the reflection and all its answers together, once. It prevents double submits (a double click, or a retry after a timeout) and half-saved records (UC-01, AC-01.2, AC-01.4). D0 says an incomplete record is never stored.

2. Inputs and outputs.

Name	Type	Notes
student_id (input)	string, UUID	The signed-in student
placement_id (input)	string, 1 to 32 characters (for example P1024)	Must belong to that student
rotation_number (input)	integer	Must be one that C4 lists as supported
rotation_source (input)	"university" or "student_supplied"	
responses (input)	array of 3 to 20 objects	Each has prompt_id (string, 1 to 32 characters), category ("accomplishment", "challenge", or "goal"), and response_text (string, 1 to 5,000 characters, at least one non-whitespace character). Limits are assumptions
reflection_id (output)	string, UUID	On success
submitted_at (output)	string, ISO 8601 UTC timestamp	On success
error (output)	invalid_response, invalid_rotation, not_authorized, duplicate_reflection, or save_failed	On failure. Nothing is stored

The steps, in order:

1. Check every answer: not empty after trimming whitespace, within the length limit, each prompt_id used once, and the set covers an accomplishment, a challenge, and a goal. Any failure rejects the whole save (AC-01.4).
2. Check that the placement belongs to the student and that the rotation number is supported.
3. Open one database transaction. Insert the REFLECTION row, then one REFLECTION_RESPONSE row per answer. Commit.
4. If the insert breaks the unique pair (placement_id, rotation_number), roll back and return duplicate_reflection with the ID of the reflection that already exists. Any other database error rolls back and returns save_failed.

3. Expected complexity. One transaction with about 5 inserts (one reflection and about 4 answers) and one unique-index check, each $O(\log n)$. At 9,000 reflections this takes a

few milliseconds. The busiest case is the end of a term. If 500 students submit in the same hour, that is less than one save per second. At 100 times that, it is about 14 per second, which one database server handles easily. We will not tune this unless a measurement shows a problem.

4. Why this one and not the alternatives.

- *Check whether a reflection exists in code, then insert:* two requests at the same moment can both pass the check and both insert.
- *Lock the student row during the save:* works, but adds locking code for something the database already does with one constraint.
- *Idempotency key from the browser:* helps with retries, but does not cover two tabs, so the constraint is still needed.
- *Save drafts as partial rows:* not in the requirements, and D0 says no incomplete record is stored. Drafts stay in the session (UC-01 E1).

A unique constraint plus one transaction is correct when requests overlap, works in any relational database, and needs very little code.

5. Edge cases.

- Empty answers array, or any whitespace-only answer: reject everything and store nothing (AC-01.4).
- Missing category, for example no goal answer: reject, because US-01 requires an improvement goal.
- Same prompt_id twice in one submission: reject.
- Double click or retry: the second request gets duplicate_reflection and the ID of the first reflection. Only one row exists (AC-01.2).
- Two tabs submit at the same moment: the database commits one and rejects the other.
- Student-supplied rotation number: stored with rotation_source set to student_supplied and not verified (AC-01.3). An unsupported number is rejected.
- Placement that does not belong to the student: reject with not_authorized.
- Failure partway through (database restart, lost connection): the transaction rolls back, and the student sees a try-again message with the answers still in the session (UC-01 E1).
- Ties: not applicable.

4. Build-versus-reuse decisions

We build only what is specific to this project: the tables, the access rule, and the save rule. Everything general (the database engine, sign-in, HTTP calls, dates, validation) is reused. These rows assume the stack in the technology section (PostgreSQL and Python). A different stack changes the library column, not the build-or-reuse split.

Piece	Build or reuse	Library or service	License	Reason (maturity, performance, fit)
Database engine for C7	Reuse	PostgreSQL	PostgreSQL License (permissive)	Decades old, and its transactions and unique constraints are exactly what Algorithm 2 needs. Fast enough for millions of rows.
Table design, constraints, indexes	Build	None	Our own	The eight tables, unique pairs, and CHECK rules come from our requirements. No library knows them.
Schema changes over time	Reuse	Alembic	MIT	Mature and widely used with SQLAlchemy. Let D2 change ships as tracked steps, not hand-run scripts.
Database access and transactions	Reuse	SQLAlchemy 2.x	MIT	Mature. Queries are parameterized, which blocks SQL injection, and it handles the transaction in Algorithm 2. Overhead is small at our size.
Database driver	Reuse	psycopg 3	LGPL-3.0	The standard PostgreSQL driver for Python. LGPL allows use as an unmodified dependency, which is how we use it. Asyncpg (Apache-2.0) is the fallback if the team wants a non-LGPL option.
Access decision (Algorithm 1)	Build	None	Our own	The rule is two checks: owner, or currently assigned advisor. Policy engines such as

Piece	Build or reuse	Library or service	License	Reason (maturity, performance, fit)
				Casbin or Open Policy Agent (both Apache-2.0) are mature but add a new tool to learn and run for a rule this small.
Save a reflection once (Algorithm 2)	Build	SQLAlchemy transaction	Our own	The logic is specific to UC-01 and sits on top of a reused transaction.
Reading the sign-in identity from E1	Reuse	Authlib (OpenID Connect) or python3-saml (SAML), (whichever UC IT supports)	BSD-3-Clause / MIT	We do not hand-roll authentication. Both are maintained and widely used.
Calls to E2 (assignment check)	Build the thin wrapper, reuse the HTTP client	httplib	BSD-3-Clause	The wrapper maps E2's responses to "assigned", "not assigned", or "unreachable". httplib gives timeouts and retries so we do not write them.
Dates and "today" in America/New_York	Reuse	Python standard library (datetime, zoneinfo)	PSF License	No hand-written date parsing. Time zone rules come from the standard library.
Request validation and API shapes	Reuse	Pydantic	MIT	Checks types, lengths, and ranges from the API contract before our code runs.

Piece	Build or reuse	Library or service	License	Reason (maturity, performance, fit)
Automated tests with seeded data	Reuse	pytest	MIT	Mature. Our INVEST self-check relies on seeded reflections and assignments, which pytest fixtures handle well.

License names are from memory of each project at the time of writing. Before we commit, each teammate should open the library's repository and confirm the license file and that the project is still maintained.

5. API contract

C2 exposes two read endpoints to C1 (the I4 link in D0). C7 is a database, so its write interface is one method that C3 calls. All timestamps are ISO 8601 in UTC. All text lengths are in characters. Every error body has the shape `{"error": {"code": string, "message": string}}`, and an error never contains reflection text.

GET /v1/reflections/{reflection_id} (C2, called by C1)

Returns one reflection if the signed-in user is its owner or its currently assigned advisor (Algorithm 1).

Input	Where	Type	Required	Valid range
reflection_id	URL path	string, UUID	Required	Canonical 36-character UUID
Authorization	Header	"Bearer" plus the token issued after E1 sign-in	Required	A token that has not expired

Output field (200 OK)	Type	Notes
reflection_id	string, UUID	
student_id	string, UUID	The owner
placement_id	string, 1 to 32 characters	For example "P1024"
rotation_number	integer, 1 or more	Unitless

Output field (200 OK)	Type	Notes
rotation_source	"university" or "student_supplied"	
submitted_at	string, ISO 8601 UTC	
responses	array of objects	Each has prompt_id (string), category ("accomplishment", "challenge", or "goal"), and response_text (string, 1 to 5,000 characters)
evaluation	object or null	Has evaluation_id (UUID), accomplishments (string), improvement_areas (string), submitted_at (ISO 8601 UTC). Only the assigned advisor receives it. Null for a student, or when no evaluation exists yet.

Error	When it happens
400 invalid_request	reflection_id is not a valid UUID
401 not_signed_in	The token is missing or expired. C1 sends the user back to E1 sign-in
403 access_denied	The user is not the owner or the currently assigned advisor, the ID does not exist, or E2 could not confirm the advisor's assignment. The body is identical in all three cases (AC-02.3)
503 backend_unavailable	The database is down. No content is returned, and the caller may try again later
500 internal_error	Any other failure. No content is returned

GET /v1/students/{student_id}/reflections (C2, called by C1)

Lists the submitted reflections of one student, so an advisor can pick one to open (UC-02 main flow). It applies the same owner-or-assigned-advisor check as Algorithm 1, using student_id directly.

Input	Where	Type	Required	Valid range
student_id	URL path	string, UUID	Required	Canonical 36-character UUID
limit	Query	integer	Optional	1 to 50, default 20
offset	Query	integer	Optional	0 or more, default 0
Authorization	Header	Bearer token	Required	Not expired

Output field (200 OK)	Type	Notes
student_id	string, UUID	
total	integer, 0 or more	Count of all submitted reflections for this student
reflections	array of objects, newest first	Each has reflection_id (UUID), rotation_number (integer), submitted_at (ISO 8601 UTC). An empty array is a normal result

Error	When it happens
400 invalid_request	student_id is not a UUID, or limit or offset is out of range
401 not_signed_in	The token is missing or expired
403 access_denied	The user is not that student or the student's currently assigned advisor, the student does not exist, or E2 could not confirm the assignment. Same body in every case
503 backend_unavailable	The database is down
500 internal_error	Any other failure

save_reflection (C7, called by C3)

Stores a completed reflection and all its answers in one transaction (Algorithm 2). It is a method call over the database connection, so its errors are codes, not HTTP statuses.

Input	Type	Required	Valid range
student_id	string, UUID	Required	The signed-in student
placement_id	string	Required	1 to 32 characters. Must belong to the student
rotation_number	integer	Required	A value C4 lists as supported
rotation_source	"university" or "student_supplied"	Required	One of the two values
responses	array of objects	Required	3 to 20 items. Each has prompt_id (1 to 32 characters, used once), category ("accomplishment", "challenge", or "goal"), and response_text (1 to 5,000 characters, at least one non-whitespace character). All three categories must appear

Output	Type	Notes
reflection_id	string, UUID	On success
submitted_at	string, ISO 8601 UTC	On success

Error code	When it happens	What the caller does
invalid_response	An answer is empty or whitespace only, too long, repeated, or a category is missing	C3 shows the missing prompts and keeps the entered text (AC-01.4). Nothing is stored
invalid_rotation	The rotation number is not one C4 lists as supported	C3 asks the student for a supported number

Error code	When it happens	What the caller does
not_authorized	The placement does not belong to the student	C3 shows an error. Nothing is stored
duplicate_reflection	A reflection already exists for this placement and rotation. The error includes the existing reflection_id	C3 shows the existing reflection as already submitted
save_failed	The database failed during the transaction	Rolled back, nothing is stored. C1 tells the student to try again

Example request and response

GET /v1/reflections/8f14e45f-ceeda-4d2a-9b7a-3c1f0e6b2a11 HTTP/1.1

Authorization: Bearer <token issued after E1 sign-in>

Accept: application/json

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "reflection_id": "8f14e45f-ceeda-4d2a-9b7a-3c1f0e6b2a11",
  "student_id": "2b7e1516-28ae-4d2a-a6a6-1d3f6c9e0b44",
  "placement_id": "P1024",
  "rotation_number": 2,
  "rotation_source": "university",
  "submitted_at": "2026-11-20T15:04:05Z",
  "responses": [
    {"prompt_id": "Q1", "category": "accomplishment", "response_text": "I improved my communication skills while working with my team."},
    {"prompt_id": "Q2", "category": "challenge", "response_text": "One challenge was learning how to use unfamiliar software."},
    {"prompt_id": "Q3", "category": "goal", "response_text": "My goal for my next co-op is to ask for feedback earlier."}
  ],
  "evaluation": null
}
```

Versioning. The API version is part of the URL path (/v1/), and adding a new endpoint or an optional response field is not a breaking change, while removing or renaming a field, changing a field's type or unit, making an optional input required, or changing what an error code means is breaking and ships as /v2 while /v1 keeps working for a notice period the team sets.

6. Technology choices with justification

Database: PostgreSQL

Team skill fit. Mbathio Lo, the team's database person, used PostgreSQL in her database theory class and SQL in her database management certificate class. She has not used it on a job yet. **Licensing.** The PostgreSQL license is permissive and free. **Community support.** It is widely used, with full official documentation and many answered questions online. **Performance.** Our data is small (about 6,000 answers now, 600,000 at 100 times), and frequent queries are indexed lookups, so responses take milliseconds. Its transactions and unique constraints are what Algorithm 2 needs. **Cost and hosting.** The software is free, and managed instances have free or low-cost tiers. We must plan backups because this is student data. **Passed over:** MongoDB, which would move the unique-pair rule and the all-or-nothing save into our own code, and SQLite, which handles several writers less well and can lose its file when a hosting container restarts.

Backend: Python 3 with FastAPI

Team skill fit. D0 lists web development, backend services, and APIs as team experience. Hinna Parwez built production Python backends on AWS (Lambda, S3, EC2) at Valmonte Digital (July to December 2024), currently builds Python AI agents at Siemens (August 2026 to present), and has written pytest suites, which matches our testing row. No one has used FastAPI, but it is a thin layer over Python and Pydantic, so the learning curve fits the semester. Python (PSF) and FastAPI, Pydantic, and SQLAlchemy (MIT) carry no fees or restrictions, and all have large communities, strong documentation, and wide use alongside our build-versus-reuse libraries. Peak load is under one save per second now and about 14 per second at 100 times, well within one small server, and async handling keeps slow E2 calls from blocking other requests. It runs in one small Docker container, which Hinna has used before. We passed over Node.js with Express, which needs add-on libraries for validation, and Django, whose built-in ORM and admin site we do not need.

Front end (C1): React

Team skill fit. Maria Malik and Priyanshi Kulshrestha have each built front ends in personal projects, and the team is more familiar with React than with the alternatives. C1 has one owner in D0, so the skill needs to be on the team, not just one person. **Licensing.** React and Vite are MIT licensed. **Community support.** React is among the most widely used front-end libraries, with extensive documentation and examples. **Performance.** C1 shows forms and short text pages, so the built files are small and load quickly on a laptop or phone, as D0 requires. The 2-second target depends on the backend, not the pages. **Cost and hosting.** The built site is static files, which many platforms host for free. **Passed over:** server-rendered pages from FastAPI templates, which are simpler to deploy but tie the pages to the backend, while D0 treats C1 as a

separate client of C2, C3, and C6, and Vue, which is comparable but less familiar to the team.

Hosting and deployment

Team skill fit. Aradana Nair brings experience working with Microsoft Azure, maintaining SAP environments, and contributing to web interface development through co-op positions. Her work with Azure included reducing data processing time and validating records between SAP and Azure, and cloud-based data workflows and troubleshooting. Her SAP Basis responsibilities included system upgrades, refreshes and recovery, which developed skills in maintaining systems and documenting technical changes. She also implemented interface updates using CSS and .NET Blazor for an internal chat-based application. Together, these experiences provide a foundation in cloud environments, system administration, and web development that can support the team's hosting and deployment work. A managed container platform with deployments from Git would allow the team to build on this foundation while limiting the infrastructure it must configure and maintain. The team will learn the selected platform's deployment process, application configuration, and monitoring tools. Prior experience deploying applications to a specific hosting platform, such as Render or Railway, remains to be confirmed with the team.

Messaging or hardware platform

Not applicable. D0 states that the system needs no special hardware, and the services call each other over HTTPS, so we need no message queue.